

Lekcja

Temat: Instrukcja DQL (ang. Data Querying Language) - SELECT, order by, LIMIT, OFFSET, WHERE, GROUP BY, having

```
DROP TABLE IF EXISTS pracownicy;
```

```
-- Tworzenie tabeli
```

```
CREATE TABLE pracownicy (  
    imie          VARCHAR(50) NOT NULL,  
    nazwisko      VARCHAR(50) NOT NULL,  
    departament  VARCHAR(50),  
    pensja       DECIMAL(10,2)  
);
```

```
-- Wstawianie danych
```

```
INSERT INTO pracownicy (imie, nazwisko, departament, pensja) VALUES  
( 'Jan',      'Kowalski',      'IT',      5000.00),  
( 'Anna',     'Nowak',          'HR',      4500.00),  
( 'Piotr',    'Zieliński',    'IT',      6200.00),  
( 'Maria',    'Wiśniewska',   'Finanse', 7000.00),  
( 'Tomasz',   'Lewandowski',  'IT',      5500.00),  
( 'Katarzyna', 'Kamińska',    'HR',      5200.00),  
( 'Adam',     'Nowicki',      'Finanse', 4800.00),  
( 'Ola',      'Dąbrowska',    'IT',      6500.00),  
( 'Robert',   'Malinowski',   'IT',      NULL);
```

ORDER BY

Instrukcja ORDER BY w zapytaniu SELECT służy do **sortowania** wyników zapytania. Bez niej wiersze są zwracane w **nieokreślonej kolejności** (zależnej od silnika bazy danych i fizycznego układu danych na dysku). ORDER BY jest **ostatnią** klauzulą w zapytaniu (przed LIMIT/OFFSET).

Składnia:

```
SELECT kolumny  
FROM tabela  
[WHERE warunek]  
ORDER BY kolumna1 [ASC|DESC], kolumna2 [ASC|DESC], ...
```

- **ASC** – rosnąco (domyślne, można pominąć)
- **DESC** – malejąco

Przykłady:

-- Pracownicy posortowani alfabetycznie po nazwisku (rosnąco)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko;
```

-- To samo, ale malejąco

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko DESC;
```

```
SELECT imie, nazwisko, pensja, pensja * 1.1 AS nowa_pensja
FROM pracownicy
ORDER BY nowa_pensja DESC;      -- można użyć aliasu
```

-- lub po pozycji kolumny w SELECT (niezalecane, ale działa)

```
SELECT imie, nazwisko, pensja, pensja * 1.1 AS nowa_pensja
FROM pracownicy
ORDER BY 3 DESC;               -- sortuje po 3. kolumnie (pensja)
```

Sortowanie z NULL

Różne bazy danych traktują *NULL* inaczej:

PostgreSQL, SQL Server, SQLite, MySQL, MariaDB - *NULL* jako najmniejsza wartość (na początku przy ASC)

Inne sposoby sortowania:

ORDER BY RANDOM() - losowa kolejność (PostgreSQL)

ORDER BY 1, 2 - sortowanie po pierwszej i drugiej kolumnie

LIMIT i OFFSET

LIMIT 3 OFFSET 4 → weź 3, pomini 4

LIMIT 3, 4 → pomini 3, weź 4

- Sposób 1 (z OFFSET – bardziej czytelny)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko
LIMIT 3 OFFSET 4;
```

-- Sposób 2 (z przecinkiem – najpopularniejszy)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko
LIMIT 4, 3;      -- UWAGA: najpierw offset, potem count!
```

WHERE

WHERE w MySQL to **filtr wierszy** – działa **przed** grupowaniem (GROUP BY) i mówi bazie: „pokaż tylko te rekordy, które spełniają warunek”.

Składnia:

```
SELECT kolumny
FROM tabela
WHERE warunek      -- ← tutaj filtrujemy
[GROUP BY ...]
[HAVING ...]
ORDER BY ...;
```

Kolejność wykonywania zapytania:

1. FROM (skąd dane)
2. WHERE ← filtrujemy wiersze
3. GROUP BY
4. HAVING
5. SELECT, ORDER BY, LIMIT

Najważniejsze operatory w WHERE

Operator	Znaczenie	Przykład
=	równa się	departament = 'IT'
!= lub <>	różne od	pensja != 5000
> / < / >= / <=	porównanie liczb	pensja > 6000
LIKE	wyszukiwanie tekstu (z % i _)	nazwisko LIKE 'K%'
IN	w liście wartości	departament IN ('IT', 'HR')
BETWEEN	w zakresie	pensja BETWEEN 5000 AND 6000
IS NULL / IS NOT NULL	NULL-e	pensja IS NULL
AND / OR / NOT	łączenie warunków	pensja > 5000 AND departament = 'IT'

Przykłady:

1. **Prosty filtr – tylko jeden dział**

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE departament = 'IT';
```

2. Więcej niż jedna warunek (AND)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE departament = 'IT'
AND pensja > 6000;
```

3. Albo jeden, albo drugi (OR)

```
SELECT imie, nazwisko, departament
FROM pracownicy
WHERE departament = 'HR'
OR departament = 'Finanse';
```

4. Wyszukiwanie tekstowe (LIKE)

```
SELECT imie, nazwisko
FROM pracownicy
WHERE nazwisko LIKE 'K%';
-- albo
WHERE nazwisko LIKE '%ski';
```

5. Zakres pensji

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE pensja BETWEEN 5000 AND 6000;
```

6. Rekordy z NULL

```
SELECT imie, nazwisko
FROM pracownicy
WHERE pensja IS NULL;
```

7. Kilka działów naraz (IN)

```
SELECT imie, nazwisko, departament
FROM pracownicy
WHERE departament IN ('IT', 'HR');
```

Różnica WHERE vs HAVING

- WHERE – filtruje **pojedyncze wiersze** (przed grupowaniem)
- HAVING – filtruje **całe grupy** (po GROUP BY)

```
-- WHERE filtruje przed grupowaniem
SELECT departament, COUNT(*)
FROM pracownicy
WHERE pensja > 5000 -- tylko osoby z pensją > 5000
GROUP BY departament;
```

```
-- HAVING filtruje po grupowaniu
SELECT departament, AVG(pensja)
FROM pracownicy
```

```
GROUP BY departament
HAVING AVG(pensja) > 5000;    -- tylko działy ze średnią > 5000
```

GROUP BY

GROUP BY w **MySQL** służy do **grupowania wierszy**, które mają te same wartości w wybranych kolumnach, a potem wykonywania na tych grupach operacji agregujących (np. liczenie, sumowanie, średnia).

Bez GROUP BY agregaty (COUNT, SUM, AVG...) **liczą wszystko razem**.

Z GROUP BY – liczą **osobno dla każdej grupy**.

Przykłady.

1. Ile osób pracuje w każdym dziale? (najpopularniejsze użycie)

```
SELECT
    departament,
    COUNT(*) AS liczba_pracownikow
FROM pracownicy
GROUP BY departament;
```

2. Średnia pensja w każdym dziale

```
SELECT
    departament,
    ROUND(AVG(pensja), 2) AS srednia_pensja,
    COUNT(*) AS ile_osob
FROM pracownicy
GROUP BY departament;
```

3. Suma wszystkich pensji w każdym dziale

```
SELECT
    departament,
    SUM(pensja) AS suma_pensji,
    MIN(pensja) AS najnizsza,
    MAX(pensja) AS najwyzsza
FROM pracownicy
GROUP BY departament;
```

4. Tylko działy, w których pracuje więcej niż 3 osoby (HAVING)

```
SELECT
    departament,
    COUNT(*) AS liczba
FROM pracownicy
GROUP BY departament
HAVING liczba > 3;    -- filtrujemy grupy!
```

5. Filtrujemy przed grupowaniem (WHERE)

```
SELECT
    departament,
    COUNT(*) AS ile,
```

```

    AVG(pensja) AS srednia
FROM pracownicy
WHERE pensja IS NOT NULL      -- pomijamy rekord z NULL
GROUP BY departament
HAVING AVG(pensja) > 5000;

```

6. Grupowanie po więcej niż jednej kolumnie

```

SELECT
    departament,
    COUNT(*) AS ile_osob
FROM pracownicy
GROUP BY departament, imie;

```

Having

Having to **klauzura** w bazach danych (nie samodzielne polecenie ani instrukcja). Jest **częścią zapytania SELECT**. Działa **tylko razem z GROUP BY**. Służy do **filtrowania całych grup** po tym, jak dane zostały zgrupowane i policzone funkcje agregujące (COUNT SUM, AVG, MAX, MIN itp.)

Składnia:

```

SELECT kolumna1, funkcja_agregująca(kolumna2)
FROM tabela
[WHERE warunek_na_wiersze]
GROUP BY kolumna1
HAVING warunek_na_grupy;

```

Porównanie HAVING z WHERE

- WHERE filtruje **pojedyncze wiersze przed** grupowaniem.
- HAVING filtruje **całe grupy po** grupowaniu.

```

CREATE TABLE IF NOT EXISTS employees (
    id            INT PRIMARY KEY,
    first_name    VARCHAR(50),
    department    VARCHAR(50),
    salary        DECIMAL(10,2)
);

```

```

INSERT INTO employees (id, first_name, department, salary) VALUES
(1, 'Jan', 'IT', 12000.00),
(2, 'Anna', 'IT', 15000.00),
(3, 'Piotr', 'IT', 8000.00),
(4, 'Maria', 'HR', 9000.00),
(5, 'Tomasz', 'HR', 11000.00),
(6, 'Kasia', 'HR', 9500.00),
(7, 'Michał', 'Sales', 20000.00),
(8, 'Ola', 'Sales', 18000.00),

```

```
(9, 'Adam', 'Marketing', 7000.00),
(10, 'Ewa', 'Marketing', 8500.00),
(11, 'Kamil', 'IT', 14000.00),
(12, 'Natalia', 'IT', 13000.00),
(13, 'Robert', 'HR', 10500.00),
(14, 'Sylwia', 'Sales', 16000.00),
(15, 'Łukasz', 'Marketing', 7500.00),
(16, 'Paweł', 'IT', 9500.00),
(17, 'Zofia', 'IT', 12500.00);
```

```
CREATE TABLE IF NOT EXISTS orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_name VARCHAR(100),
    price DECIMAL(10,2),
    quantity INT,
    total_amount DECIMAL(10,2)
);
```

```
INSERT INTO orders (order_id, customer_id, product_name, price, quantity, total_amount)
VALUES
(1, 101, 'Laptop', 4500.00, 1, 4500.00),
(2, 101, 'Smartphone', 2500.00, 1, 2500.00),
(3, 101, 'Headphones', 300.00, 2, 600.00),
(4, 101, 'Laptop', 4200.00, 1, 4200.00),
(5, 101, 'Monitor', 1200.00, 1, 1200.00),
(6, 102, 'Laptop', 4300.00, 1, 4300.00),
(7, 102, 'Smartphone', 2800.00, 1, 2800.00),
(8, 102, 'Tablet', 1800.00, 1, 1800.00),
(9, 102, 'Laptop', 4600.00, 1, 4600.00),
(10, 103, 'Headphones', 250.00, 3, 750.00),
(11, 103, 'Smartphone', 2200.00, 1, 2200.00),
(12, 104, 'Monitor', 1100.00, 1, 1100.00),
(13, 101, 'Tablet', 1700.00, 1, 1700.00),
(14, 105, 'Laptop', 5000.00, 1, 5000.00),
(15, 102, 'Headphones', 350.00, 2, 700.00),
(16, 101, 'Monitor', 1300.00, 1, 1300.00),
(17, 106, 'Smartphone', 2400.00, 1, 2400.00),
(18, 101, 'Laptop', 4800.00, 1, 4800.00),
(19, 102, 'Tablet', 1900.00, 1, 1900.00),
(20, 103, 'Monitor', 1150.00, 1, 1150.00);
```

Przykłady

1: Liczba pracowników w departamentach (tylko te z > 5 osobami)

```
SELECT
    department,
    COUNT(*) AS liczba_pracownikow
FROM employees
```

```
GROUP BY department
HAVING COUNT(*) > 5;
```

2: Produkty, które sprzedały się za więcej niż 10 000 zł

```
SELECT
    product_name,
    SUM(price * quantity) AS suma_sprzedazy
FROM orders
GROUP BY product_name
HAVING SUM(price * quantity) > 10000;
```

3: Klienci, którzy złożyli więcej niż 3 zamówienia (i średnia wartość zamówienia > 500)

```
SELECT
    customer_id,
    COUNT(order_id) AS liczba_zamowien,
    AVG(total_amount) AS srednia_wartosc
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) > 3
    AND AVG(total_amount) > 500;
```

4: Działy, w których maksymalna pensja przekracza 15 000 zł

```
SELECT
    department,
    MAX(salary) AS max_pensja
FROM employees
GROUP BY department
HAVING MAX(salary) > 15000;
```

5: Departamenty z więcej niż 5 pracownikami

```
SELECT
    department,
    COUNT(*) AS liczba_pracownikow
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

6: Produkty, które sprzedały się za więcej niż 10 000 zł

```
SELECT
    product_name,
    SUM(price * quantity) AS suma_sprzedazy
FROM orders
GROUP BY product_name
HAVING SUM(price * quantity) > 10000;
```

7: Klienci z > 3 zamówieniami i średnią wartością zamówienia > 500 zł

```
SELECT
    customer_id,
    COUNT(order_id) AS liczba_zamowien,
    AVG(total_amount) AS srednia_wartosc
FROM orders
```



```
GROUP BY customer_id
HAVING COUNT(order_id) > 3
      AND AVG(total_amount) > 500;
```

8: Działy, w których maksymalna pensja przekracza 15 000 zł

```
SELECT
    department,
    MAX(salary) AS max_pensja
FROM employees
GROUP BY department
HAVING MAX(salary) > 15000;
```

Lekcja

Temat: Typy danych w MySQL – typy liczbowe

Materiały do lekcji wzorowane na dokumentacji MySQL:

<https://dev.mysql.com/doc/refman/8.4/en/data-types.html>

MySQL obsługuje typy danych w kilku kategoriach:

- typy liczbowe,
- typy daty i godziny,
- typy ciągów znaków (znakowe i bajtowe),
- typy przestrzenne
- typy danych JSON

Typy danych liczbowych

MySQL obsługuje wszystkie standardowe typy danych liczbowych zgodne z SQL.

Dzielą się one na dokładne typy danych liczbowych:

- TINYINT
- SMALLINT
- MEDIUMINT
- INTEGER synonim INT,
- BIGINT,

- [DECIMAL](#) synonim DEC i FIXED,
- [NUMERIC](#),

przybliżone typy danych liczbowych:

- [FLOAT](#),
- REAL synonim [DOUBLE PRECISION](#),
- DOUBLE synonim [DOUBLE PRECISION](#).

oraz typ wartości bitowej

- BIT - służy do przechowywania wartości bitowych i jest obsługiwany w tabelach typu MyISAM, MEMORY, InnoDB oraz NDB.

Typ	Rozmiar (bajty)	Minimalna wartość (ze znakiem) Signed	Maksymalna wartość (ze znakiem) Signed	Minimalna wartość (bez znaku) Unsigned	Maksymalna wartość (bez znaku) Unsigned
TINYINT	1	-128	127	0	255
SMALLINT	2	-32768	32767	0	65535
MEDIUMINT	3	-8388608	8388607	0	16777215
INT	4	-2147483648	2147483647	0	4294967295
BIGINT	8	-2^{63}	$2^{63} - 1$	0	$2^{64} - 1$

Polecenie zwracające informację o trybie ścisłym (strict mode) w MySQL.

```
SELECT @@sql_mode;
```

albo dokładniej:

```
SHOW VARIABLES LIKE 'sql_mode';
```

```
SHOW GLOBAL VARIABLES LIKE 'sql_mode';
```

🔗 Jeśli w wyniku zobaczysz np.:

- STRICT_TRANS_TABLES
- lub STRICT_ALL_TABLES

to znaczy, że tryb ścisły jest **włączony**.

Dodatkowe przydatne polecenia

Polecenie	zwraca
SELECT @@GLOBAL.sql_mode;	Tryb globalny (dla całego serwera)
SELECT @@SESSION.sql_mode;	Tryb dla Twojego aktualnego połączenia
SELECT @@sql_mode;	Skrót do sesji

Nie ma STRICT_TRANS_TABLES ani STRICT_ALL_TABLES → tryb **nieścisły**

Różnice między STRICT_TRANS_TABLES a STRICT_ALL_TABLES

👉 STRICT_TRANS_TABLES

- tryb ścisły działa **tylko dla tabel transakcyjnych** (np. InnoDB)
- dla innych tabel (np. MyISAM) MySQL może **zamiast błędu dać ostrzeżenie i poprawić dane**

👉 STRICT_ALL_TABLES

- tryb ścisły działa **dla wszystkich tabel**
- zawsze dostajesz **błąd**, niezależnie od typu tabeli

Zmiana na tryb ścisły dla obecnej sesji

```
SET SESSION sql_mode = CONCAT(@@sql_mode, ',STRICT_TRANS_TABLES');
```

To automatycznie **zachowa** wszystkie Twoje obecne ustawienia i tylko doda tryb ścisły.

Weryfikacja, czy się zmienił na tryb ścisły

```
SELECT @@SESSION.sql_mode;
```

Ustawienie trybu ścisłego (Strict Mode)

GLOBAL (dla całego serwera – wszystkie nowe połączenia)

- **Tylko STRICT_TRANS_TABLES** (zalecany w większości przypadków):

```
SET GLOBAL sql_mode = 'STRICT_TRANS_TABLES';
```

- **Tylko STRICT_ALL_TABLES** (najsurowszy – działa dla wszystkich tabel, w tym MyISAM):

```
SET GLOBAL sql_mode = 'STRICT_ALL_TABLES';
```

SESSION (tylko dla aktualnego połączenia)

- **STRICT_TRANS_TABLES:**

```
SET SESSION sql_mode = 'STRICT_TRANS_TABLES';
```

- **STRICT_ALL_TABLES:**

```
SET SESSION sql_mode = 'STRICT_ALL_TABLES';
```

Dodanie trybu ścisłego bez nadpisywania innych ustawień (zalecane!)

❖ **GLOBAL**

```
SET GLOBAL sql_mode = CONCAT(@@GLOBAL.sql_mode, ',STRICT_TRANS_TABLES');  
SET GLOBAL sql_mode = CONCAT(@@GLOBAL.sql_mode, ',STRICT_ALL_TABLES');
```

❖ **SESSION**

```
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, ',STRICT_TRANS_TABLES');  
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, ',STRICT_ALL_TABLES');
```

Wyłączenie trybu ścisłego (usunięcie konkretnego trybu)

GLOBAL

```
SET GLOBAL sql_mode = REPLACE(REPLACE(@@GLOBAL.sql_mode,  
'STRICT_TRANS_TABLES', ''), ',, ', ', ');  
SET GLOBAL sql_mode = REPLACE(REPLACE(@@GLOBAL.sql_mode, 'STRICT_ALL_TABLES',  
''), ',, ', ', ');
```

SESSION

```
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode,  
'STRICT_TRANS_TABLES', ''), ',, ', ', ');  
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode,  
'STRICT_ALL_TABLES', ''), ',, ', ', ');
```

Przykład

```
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, ',STRICT_ALL_TABLES');  
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode, 'STRICT_ALL_TABLES', ''),  
'', ',, ', ', ');  
CREATE TABLE test_int_UNSIGNED (  
    liczba      INT UNSIGNED  
) ENGINE=InnoDB;  
  
INSERT INTO test_int_UNSIGNED (liczba) VALUES (4294967296 );  
SELECT * from test_int_UNSIGNED;  
  
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, ',STRICT_ALL_TABLES');  
SET SESSION sql_mode = REPLACE(REPLACE(@@SESSION.sql_mode, 'STRICT_ALL_TABLES', ''),  
'', ',, ', ', ');  
  
CREATE TABLE test_int_SIGNED (  
    liczba      INT SIGNED  
) ENGINE=InnoDB;  
  
INSERT INTO test_int_SIGNED (liczba) VALUES (-2147483999 );  
SELECT * from test_int_SIGNED;
```

Tryb	Co się dzieje z danymi?	Komunikat MySQL	Efekt INSERT / UPDATE
Tryb nieściśły (sql_mode pusty lub bez STRICT_*)	Obcina wartość do najbliższej dopuszczalnej (np. maksimum lub 0) i zapisuje ją	Warning (ostrzeżenie) nr 1264 „Out of range value for column...”	Operacja kończy się sukcesem (Query OK)
Tryb ściśły (STRICT_TRANS_TABLES lub STRICT_ALL_TABLES)	Odrzuca całą operację – wartość nie jest zapisywana	ERROR 1264 (22003) „Out of range value for column...”	Operacja kończy się błędem (INSERT/UPDATE nie przechodzi)

- **Tryb ściśły (Strict SQL mode)**

Najbezpieczniejszy — MySQL zachowuje się zgodnie ze standardem SQL.

Każda wartość spoza zakresu powoduje błąd i nie wstawia żadnego wiersza (lub przerywa operację).

Zalecany w nowych projektach.

- **Tryb luźny (non-strict)**

MySQL nie przerywa operacji, tylko „cicho” przycina wartości do granicy zakresu.

Zamiast błędu generuje tylko **warning** (ostrzeżenie).

Może prowadzić do utraty danych bez Twojej wiedzy — dlatego wielu programistów go nie lubi.

ZEROFILL to **atrybut wyświetlania** (display attribute) w MySQL i MariaDB dla typów liczbowych (głównie INTEGER, TINYINT, SMALLINT, MEDIUMINT, BIGINT, a także FLOAT, DOUBLE, DECIMAL).

Działa tak:

- Gdy liczba ma **mniej cyfr** niż określona szerokość wyświetlenia, MySQL **dopełnia ją zerami z lewej strony**.
- Jest to **tylko kwestia wyświetlania** – w bazie danych wartość jest przechowywana normalnie (bez zer).

Ważne fakty:

- Gdy dodasz ZEROFILL, MySQL **automatycznie** dodaje atrybut UNSIGNED (kolumna nie może przechowywać liczb ujemnych).

- Atrybut ZEROFILL jest **przestarzały** (deprecated) w nowszych wersjach MySQL 8.0+ i MariaDB. W przyszłości może zostać całkowicie usunięty. Zaleca się go unikać.

Przykłady działania

```
CREATE TABLE test_zerofill (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nr1         INT(5) ZEROFILL,
  nr2         TINYINT(3) ZEROFILL,
  nr3         INT(8) ZEROFILL
);
```

```
INSERT INTO test_zerofill (nr1, nr2, nr3) VALUES
(7, 42, 123),
(42, 5, 9876543),
(123, 100, 1);
```

```
SELECT * FROM test_zerofill;
```

id	nr1	nr2	nr3
1	00007	042	00000123
2	00042	005	09876543
3	00123	100	00000001

BIT[(M)]

Typ bitowy. M określa liczbę bitów na wartość — od **1 do 64**. Jeśli M zostanie pominięte, domyślna wartość wynosi **1**.

TINYINT[(M)] [UNSIGNED] [ZEROFILL]

Bardzo mała liczba całkowita.

- **Zakres ze znakiem (signed):** -128 do 127
- **Zakres bez znaku (unsigned):** 0 do 255

BOOL, BOOLEAN

Są to synonimy dla TINYINT(1). Wartość **zero** jest traktowana jako **false**, a każda wartość różna od zera jako **true**.

```
SELECT IF(0, 'true', 'false');  -- wynik: false
SELECT IF(1, 'true', 'false');  -- wynik: true
SELECT IF(2, 'true', 'false');  -- wynik: true
```

Wartości TRUE i FALSE są jedynie aliasami dla 1 i 0:

```
SELECT IF(0 = FALSE, 'true', 'false');  -- true, bo 0=0 (false daje 0)
SELECT IF(1 = TRUE, 'true', 'false');   -- true
SELECT IF(2 = TRUE, 'true', 'false');   -- false, bo 2=1 daje false
SELECT IF(2 = FALSE, 'true', 'false');  -- false
```

Uwaga: Wartość 2 nie jest równa ani TRUE (1), ani FALSE (0).

SMALLINT[(M)] [UNSIGNED] [ZEROFILL]

Mała liczba całkowita.

- **Zakres ze znakiem:** -32 768 do 32 767
- **Zakres bez znaku:** 0 do 65 535

MEDIUMINT[(M)] [UNSIGNED] [ZEROFILL]

Średnia liczba całkowita.

- **Zakres ze znakiem:** -8 388 608 do 8 388 607
- **Zakres bez znaku:** 0 do 16 777 215

INT[(M)] [UNSIGNED] [ZEROFILL]

Zwykła liczba całkowita (standardowa).

- **Zakres ze znakiem:** -2 147 483 648 do 2 147 483 647
- **Zakres bez znaku:** 0 do 4 294 967 295

INTEGER[(M)] [UNSIGNED] [ZEROFILL]

Synonim typu INT.

BIGINT[(M)] [UNSIGNED] [ZEROFILL]

Duża liczba całkowita.

- **Zakres ze znakiem:** -9 223 372 036 854 775 808 do 9 223 372 036 854 775 807
- **Zakres bez znaku:** 0 do 18 446 744 073 709 551 615

SERIAL jest aliasem dla: BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE

Typy DECIMAL i NUMERIC służą do przechowywania **dokładnych wartości liczbowych**. Są używane w sytuacjach, gdy kluczowe jest zachowanie **pełnej precyzji**, na przykład przy danych finansowych (pieniężnych).

W MySQL typ NUMERIC jest w pełni zaimplementowany jako DECIMAL, dlatego wszystkie informacje dotyczące DECIMAL dotyczą również NUMERIC.

Deklaracja kolumny DECIMAL

Przy tworzeniu kolumny typu DECIMAL można (i zwykle należy) określić **precyzję** oraz **skalę**.

Alias

DEC / NUMERIC / FIXED = DECIMAL

Przykład:

salary **DECIMAL**(5,2)

- **5** — to **precyzja** (precision) — całkowita liczba znaczących cyfr, które mogą być przechowywane.
- **2** — to **skala** (scale) — liczba cyfr po przecinku dziesiętnym.

Przykład:

```
CREATE TABLE test_decimal (  
    salary DECIMAL(5,2)  
) ENGINE=InnoDB;
```

```
INSERT INTO test_decimal (salary) VALUES (-999.99 );  
INSERT INTO test_decimal (salary) VALUES (999.99 );  
INSERT INTO test_decimal (salary) VALUES (9991.2);  
INSERT INTO test_decimal (salary) VALUES (-1000.50);  
INSERT INTO test_decimal (salary) VALUES (1000.00);  
INSERT INTO test_decimal (salary) VALUES (123.456 );  
INSERT INTO test_decimal (salary) VALUES (99910.999);  
SELECT * from test_decimal;
```

Wynik:

salary

```
-999.99  
  
999.99  
  
999.99  
  
-999.99  
  
999.99  
  
123.46  
  
999.99
```

Kolumna z maksymalną precyzją **DECIMAL**(65,30)

- **65** → maksymalna liczba **wszystkich cyfr** (przed i po przecinku)
- **30** → maksymalna liczba cyfr **po przecinku**

Przykład: Z przekroczonym zakresem precyzji **DECIMAL(66,30)**

```
CREATE TABLE test_max_decimal (  
    bardzo_duza_liczba DECIMAL(66,31)    -- maksymalna możliwa precyzja  
) ENGINE=InnoDB;
```


Error

Zapytanie SQL: [Copy](#)

```
CREATE TABLE test_max_decimal2 (
    bardzo_duza_liczba DECIMAL(66,31),    -- maksymalna możliwa precyzja
) ENGINE=InnoDB;
```

MySQL zwrócił komunikat:

```
#1426 - Too big precision 66 specified for 'bardzo_duza_liczba'. Maximum is 65
```

Przykład:

```
CREATE TABLE test_max_decimal5(
    bardzo_duza_liczba DECIMAL(65,30)
) ENGINE=InnoDB;
```

[illegible]

```
Select * from test_max_decimal5;
```

bardzo duza liczba

[illegible]

-0.0000000000000000000000000000000001

12345678901234567890123456789012345.678901234567890123456789012345

[illegible][illegible][illegible]

Typy FLOAT i DOUBLE służą do przechowywania **przybliżonych wartości liczbowych** (w przeciwieństwie do DECIMAL, który przechowuje wartości dokładnie).

MySQL używa:

- **4 bajtów** dla typu FLOAT (pojedyncza precyzja – single precision)
- **8 bajtów** dla typu DOUBLE (podwójna precyzja – double precision)

Specyfikacja precyzji według standardu SQL

Standard SQL pozwala na opcjonalne podanie precyzji w bitach po słowie FLOAT:

FLOAT(p)

MySQL obsługuje tę składnię, ale traktuje parametr p wyłącznie jako informację o rozmiarze przechowywania:

- **p od 0 do 23** → kolumna typu FLOAT (4 bajty, pojedyncza precyzja)
- **p od 24 do 53** → kolumna typu DOUBLE (8 bajty, podwójna precyzja)

MySQL pozwala na starszą, **niezalecaną** składnię:

FLOAT(M,D)

DOUBLE(M,D)

REAL(M,D) -- REAL jest synonimem DOUBLE

DOUBLE PRECISION(M,D)

Gdzie:

- **M** = maksymalna liczba **wszystkich cyfr** (przed i po przecinku)
- **D** = liczba cyfr **po przecinku**

Przy zapisie MySQL wykonuje **zaokrąglanie**. Jeśli wstawisz 999.00009 do kolumny FLOAT(7,4), zostanie zapisane jako **999.0001**.

Ważna uwaga:

FLOAT(M,D) oraz DOUBLE(M,D) jest **niezgodna ze standardem** i **przestarzała** (deprecated). W przyszłości wsparcie dla tych wariantów może zostać całkowicie usunięte z MySQL.

Nie zaleca się ich używania w nowych projektach.

Najlepsza praktyka (zalecana)

Dla maksymalnej przenośności kodu zaleca się używanie:

FLOAT -- lub FLOAT(p)

DOUBLE -- lub DOUBLE PRECISION

bez podawania precyzji (M,D) ani liczby cyfr.

Typ danych BIT służy do przechowywania **wartości bitowych** (ciągów zer i jedynek).

Składnia:

BIT(M)

- **M** określa liczbę bitów, które mogą być przechowywane w kolumnie.
- **M** może wynosić od **1 do 64**.

Do wstawiania wartości bitowych służy notacja **b'value**, gdzie value składa się wyłącznie z cyfr **0** i **1**.

Przykłady:

- b'111' → oznacza liczbę **7** (w systemie dziesiętnym)
- b'10000000' → oznacza liczbę **128**
- b'0' → 0
- b'1' → 1
- b'1010' → 10

Dopełnianie zerami (padding)

Jeśli przypiszesz do kolumny BIT(M) wartość, która ma **mniej niż M bitów**, MySQL automatycznie **dopełni ją zerami z lewej strony**.

Przykład:

Przypisanie wartości b'101' do kolumny BIT(6) jest równoważne przypisaniu b'000101'.

Czyli:

- b'101' (3 bity) → automatycznie staje się b'000101' (6 bitów)

MySQL obsługuje rozszerzenie pozwalające **opcjonalnie określić szerokość wyświetlania** (display width) dla typów całkowitoliczbowych. Szerokość podaje się w nawiasach po nazwie typu.

Przykład:

INT(4)

oznacza typ INT z szerokością wyświetlania równą 4 cyfrom.

Do czego służy szerokość wyświetlania?

Szerokość wyświetlania jest używana przez aplikacje do formatowania wyników. Jeśli wartość ma mniej cyfr niż określona szerokość, aplikacja może **dopełnić ją spacjami z lewej strony**. Ta informacja jest zwracana w metadanych wyniku zapytania — czy zostanie faktycznie użyta, zależy wyłącznie od aplikacji.

Bardzo ważne:

- Szerokość wyświetlania **nie ogranicza** zakresu wartości, jakie można zapisać w kolumnie.
- Nie zapobiega też poprawnemu wyświetlaniu większych liczb.

Przykład: Kolumna SMALLINT(3) nadal ma pełny zakres od -32768 do 32767. Wartości spoza 3 cyfr (np. 12345) będą wyświetlane w całości, z większą liczbą cyfr.

```
CREATE TABLE test_SMALLINT(  
    mala_liczba SMALLINT(3)  
) ENGINE=InnoDB;
```

```
INSERT INTO test_SMALLINT ( mala_liczba ) VALUES  
    ( 12345),  
    ( 12.1 );  
Select * from test_SMALLINT;
```

<u>mala_liczba</u>
12345
12

Atrybut ZEROFILL

Gdy użyjemy atrybutu ZEROFILL razem z szerokością wyświetlania, zamiast spacji wartości są dopełniane **zerami** z lewej strony.

Przykład:

INT(4) ZEROFILL

Wartość 5 zostanie zwrócona jako **0005**.

Uwaga: Atrybut ZEROFILL jest ignorowany, gdy kolumna jest używana w wyrażeniach lub zapytaniach UNION.

Przepelnienie (Overflow) podczas obliczeń

Przykład z największą wartością BIGINT (ze znakiem):

```
SELECT 9223372036854775807 + 1;    -- błąd
```

Rozwiązania:

- Rzutowanie na UNSIGNED:

```
SELECT CAST(9223372036854775807 AS UNSIGNED) + 1; -- zwraca  
9223372036854775808
```

- Użycie arytmetyki dokładnej (DECIMAL):

```
SELECT 9223372036854775807.0 + 1;
```

Lekcja

Temat: Typy danych w MySQL – daty i godziny, typy ciągów znaków (znakowe i bajtowe)

Materialy do lekcji wzorowane na dokumentacji MySQL:

<https://dev.mysql.com/doc/refman/8.4/en/date-and-time-types.html>

MySQL obsługuje typy danych w kilku kategoriach:

- typy liczbowe,
- typy daty i godziny,
- typy ciągów znaków (znakowe i bajtowe),
- typy przestrzenne
- typy danych JSON

Typy danych daty i godziny

MySQL udostępnia 5 głównych typów do przechowywania wartości czasowych:

- DATE – tylko data (YYYY-MM-DD)
- TIME – tylko czas (HH:MM:SS)
- DATETIME – data + czas
- TIMESTAMP – data + czas (z dodatkowymi właściwościami)

- YEAR – rok

MySQL akceptuje następujące formaty:

- **Z separatorami** (zalecany): 'YYYY-MM-DD' lub 'YY-MM-DD' Przykład: '2025-12-31', '25-12-31'
- **„Luźna” składnia** (dowolny znak interpunkcyjny jako separator) jest deprecated w MySQL 8.4 i generuje warning 4095) Dowolny znak interpunkcyjny jako separator, np. '2012/12/31', '2012^12^31', '2012@12@31'. Zalecany jest zawsze myślnik -.
- **Bez separatorów** (ciąg): 'YYYYMMDD' lub 'YYMMDD' Przykład: '20251231' → '2025-12-31', '251231' → '2025-12-31'
- **Jako liczba**: YYYYMMDD lub YYMMDD Przykład: 20251231 lub 251231

MySQL interpretuje lata 2-cyfrowe tak:

- 70-99 → 1970-1999
- 00-69 → 2000-2069

👉 dlatego lepiej używać zawsze **4 cyfr**

```
CREATE TABLE test_daty (
  id INT AUTO_INCREMENT PRIMARY KEY,
  data DATE NOT NULL
);
```

-- Wstawiamy daty z dwucyfrowym rokiem (format YY-MM-DD)

```
INSERT INTO test_daty (data) VALUES
  ('69-12-31'), -- 00-69 → 2000-2069 → 2069-12-31
  ('70-01-01'), -- 70-99 → 1970-1999 → 1970-01-01
  ('00-01-01'), -- 00-69 → 2000-2069 → 2000-01-01
  ('99-12-31'); -- 70-99 → 1970-1999 → 1999-12-31
```

-- Sprawdzamy, jak MySQL je zapamiętał

```
SELECT
  id,
  data,
  YEAR(data) AS rok
FROM test_daty;
```

Oczekiwany wynik:

id	data	rok
1	2069-12-31	2069
2	1970-01-01	1970
3	2000-01-01	2000
4	1999-12-31	1999

Do konwersji użyj funkcji:

STR_TO_DATE()

```
SELECT
    '69-12-31' AS tekst_wejsciowy,
    STR_TO_DATE('69-12-31', '%y-%m-%d') AS data_z_y,
    YEAR(STR_TO_DATE('69-12-31', '%y-%m-%d')) AS rok_z_y,

    STR_TO_DATE('70-01-01', '%y-%m-%d') AS data_z_y_70,
    YEAR(STR_TO_DATE('70-01-01', '%y-%m-%d')) AS rok_z_y_70,

    STR_TO_DATE('00-01-01', '%y-%m-%d') AS data_z_y_00,
    YEAR(STR_TO_DATE('00-01-01', '%y-%m-%d')) AS rok_z_y_00,

    STR_TO_DATE('99-12-31', '%y-%m-%d') AS data_z_y_99,
    YEAR(STR_TO_DATE('99-12-31', '%y-%m-%d')) AS rok_z_y_99;
```

Nieprawidłowe wartości

Sprawdzenie aktualnego trybu

```
SELECT @@sql_mode;
```

Domyślnie w MySQL 8.x wygląda mniej więcej tak:

ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_F
OR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION

Przygotowanie tabeli testowej

```
CREATE TABLE test_data (
    id INT AUTO_INCREMENT PRIMARY KEY,
    d DATE,
    dt DATETIME,
    t TIME
);
```

A. Tryb nieściśle (non-strict)

```
CREATE TABLE test_data3 (
    id INT AUTO_INCREMENT PRIMARY KEY,
    d DATE,
    dt DATETIME,
```

```
t    TIME
);
```

```
SET SESSION sql_mode =
'ONLY_FULL_GROUP_BY,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION,ALLOW_INVALID_DATES';
```

```
INSERT INTO test_data3 (d, dt, t) VALUES
('2023-02-30', '2023-13-05 25:70:99', '999:99:99'),
('0000-00-00', '0000-00-00 00:00:00', '-1000:00:00');
```

```
SELECT * FROM test_data3;
```

Wynik:

- DATE i DATETIME → zamieniane na '0000-00-00' / '0000-00-00 00:00:00' + **warning** (nie błąd)
- TIME → **obcinany** do granic zakresu:
 - o 999:99:99 → '838:59:59'
 - o -1000:00:00 → '-838:59:59'

B. Tryb ścisły (strict) – zalecany w nowych projektach

```
CREATE TABLE test_data4 (
  id    INT AUTO_INCREMENT PRIMARY KEY,
  d      DATE,
  dt     DATETIME,
  t      TIME
);
```

```
SET SESSION sql_mode =
'STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
```

```
INSERT INTO test_data4 (d, dt, t) VALUES
('2023-02-30', '2023-13-05 25:70:99', '999:99:99'),
('0000-00-00', '0000-00-00 00:00:00', '-1000:00:00');
```

```
SELECT * FROM test_data4;
```


Error

Zapytanie SQL: [Copy](#)

```
INSERT INTO test_data4 (d, dt, t) VALUES  
( '2023-02-30', '2023-13-05 25:70:99', '999:99:99'),  
( '0000-00-00', '0000-00-00 00:00:00', '-1000:00:00');
```

MySQL zwrócił komunikat:

```
#1292 - Incorrect date value: '2023-02-30' for column `data-tryby`.`test_data4`.`d` at row 1
```

Wynik:

- Wszystkie niepoprawne daty (2023-02-30, 2023-13-05, '0000-00-00') → **błąd** (Error 1292 lub 1525: Incorrect date/datetime value)
- Niepoprawny czas (999:99:99, -1000:00:00) → **błąd** (out of range)
- Cała instrukcja INSERT kończy się niepowodzeniem (dla tabel transakcyjnych – rollback, dla MyISAM może być częściowy insert).

⚙ Tryby SQL

1. ALLOW_INVALID_DATES – pozwala na „głupie” daty

Wyłącza **pełną walidację dat**. MySQL sprawdza tylko:

- miesiąc między 1 a 12,
- dzień między 1 a 31.

Nie sprawdza rzeczy takich jak:

- luty ma tylko 28/29 dni,
- kwiecień, czerwiec, wrzesień, listopad mają tylko 30 dni,
- rok przestępny itp.

Przykład co pozwala:

- '2009-11-31' ← listopad ma tylko 30 dni → normalnie błąd, z tym trybem przechodzi
- '2023-02-30'
- '2024-04-31'
- '0000-00-00' (w połączeniu z innymi trybami)

Kiedy się przydaje

- Importujesz stare, brudne dane z legacy systemów

- Aplikacja zbiera rok, miesiąc i dzień z osobnych pól formularza i chcesz zapisać dokładnie to, co wpisał użytkownik (nawet jeśli data jest bez sensu)
- Migracja danych

Uwaga: Ten tryb **nie wpływa** na TIME i nie wyłącza wszystkich kontroli (np. rok musi być sensowny).

2. NO_ZERO_DATE – blokuje pełną zerową datę

Blokuje

- '0000-00-00'
- '0000-00-00 00:00:00'

Zachowanie:

- **Bez trybu strict** → tylko warning (data może być wstawiona)
- **Z trybem strict** (STRICT_TRANS_TABLES lub STRICT_ALL_TABLES) → **błąd** (insert się nie uda)

Zerowa data to specjalna wartość MySQL oznaczająca „brak daty”. W nowych projektach lepiej używać NULL zamiast '0000-00-00'.

Uwaga: Od MySQL 5.7+ ten tryb jest **deprecated** (wycofywany), ale nadal działa. W przyszłości jego rolę przejmie sam strict mode.

3. NO_ZERO_IN_DATE – blokuje częściowe zera w dacie

Blokuje

- '2009-00-00' (miesiąc = 0)
- '2009-05-00' (dzień = 0)
- '0000-00-15' (miesiąc i dzień = 0, ale rok niezerowy)
- '2023-00-15'

Nie blokuje:

- '0000-00-00' (to kontroluje NO_ZERO_DATE)

Zachowanie w strict mode: błąd **Bez strict:** warning + zamiana na zero w niektórych przypadkach

Takie daty prawie zawsze oznaczają błąd w danych. Ten tryb pomaga utrzymać czystość bazy.

Uwaga: Tryby **NO_ZERO_DATE** i **NO_ZERO_IN_DATE** są oznaczone jako deprecated od MySQL 8.4. W przyszłości ich efekt będzie częścią strict mode.”

MySQL pozwala przechowywać „puste” daty bez strict. Lepiej zastosować NULL:

Typ	Wartość zero
DATE	'0000-00-00'
TIME	'00:00:00'
DATETIME	'0000-00-00 00:00:00'
TIMESTAMP	'0000-00-00 00:00:00'
YEAR	0000

Ułamkowe sekundy

MySQL obsługuje milisekundy:

DATETIME(3) -- milisekundy
 DATETIME(6) -- mikrosekundy
 TIME(6)
 TIMESTAMP(6)

MySQL używa proleptycznego kalendarza gregoriańskiego (ang. proleptic Gregorian calendar).

- **Kalendarz gregoriański** – to ten, którego używamy na co dzień od 1582 roku (wprowadzony przez papieża Grzegorza XIII). Ma zasady:
 - Lata przestępne co 4 lata,
 - Ale nie co 100 lat (chyba że podzielne przez 400),
 - Miesiące mają stałą liczbę dni (np. luty 28 lub 29, kwiecień 30 itd.).
- **Proleptyczny** = „wyprzedzający” / „rozciągnięty wstecz”. Oznacza, że MySQL **zawsze** stosuje **reguły gregoriańskie**, nawet do dat sprzed 1582 roku (kiedy w rzeczywistości jeszcze nie istniał kalendarz gregoriański – wtedy był kalendarz juliański).

Czyli MySQL **udaje**, że kalendarz gregoriański istniał od zawsze (od roku 0001 lub nawet wcześniej

Zakres przechowywanych wartości

- **DATETIME**: od '1000-01-01 00:00:00' do '9999-12-31 23:59:59'
- **TIMESTAMP**: od '1970-01-01 00:00:01' UTC do '2038-01-19 03:14:07' UTC (problem roku 2038)
- **DATETIME**: **nie uwzględnia strefy czasowej**. Przechowuje wartość dosłownie taką, jaką wpiszesz.
- **TIMESTAMP**: **zawsze przechowuje wartości w UTC**. Przy zapisie konwertuje z lokalnej strefy serwera na UTC, przy odczycie odwrotnie.

UTC (ang. **Coordinated Universal Time**) to **uniwersalny czas koordynowany** – międzynarodowy standard czasu, który służy jako **wspólny punkt odniesienia** dla całego świata.

UTC to „czas zerowy” – taki sam w każdej chwili na całej Ziemi. Nie ma w nim czasu letniego (DST) ani zmian sezonowych. Wszystkie strefy czasowe na świecie są liczone właśnie względem UTC.

Przykład:

- Polska w zimie: **UTC +1** (czas środkowoeuropejski – CET)
- Polska w lecie: **UTC +2** (czas letni – CEST)
- Londyn (zimą): **UTC +0**
- Nowy Jork: **UTC -4** lub **UTC -5** (w zależności od pory roku)

W MySQL typ **TIMESTAMP** działa właśnie w oparciu o **UTC**:

- Gdy zapisujesz wartość do kolumny **TIMESTAMP**, MySQL **automatycznie konwertuje** ją z Twojej strefy czasowej sesji **na UTC** i tak przechowuje.
- Gdy odczytujesz – konwertuje z powrotem na strefę czasową aktualnej sesji.
- Typ **DATETIME** **nie robi** tej konwersji – co zapiszesz, to odczytasz (niezależny od strefy)

```
CREATE TABLE test_tz (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  dt          DATETIME,           -- nie zależy od strefy  
  ts          TIMESTAMP,         -- zależy od strefy (przechowywany w UTC)  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Test zachowania:

```
-- 1. Ustawiamy strefę na Polskę (CEST = UTC+2)  
SET time_zone = '+02:00';  
  
INSERT INTO test_tz (dt, ts)  
VALUES ('2026-04-15 12:00:00', '2026-04-15 12:00:00');  
  
SELECT dt, ts, created_at, updated_at FROM test_tz;
```

Wynik (przykład):

- dt → 2026-04-14 09:45:30 ← dokładnie to, co wstawiono
- ts → 2026-04-14 09:45:30 ← pokazane w strefie sesji (+2)
- created_at i updated_at → podobnie

Teraz zmień strefę i odczytaj ponownie:

```
SET time_zone = '+00:00';
SELECT dt, ts, created_at, updated_at FROM test_tz;
```

Wynik:

- dt → 2026-04-14 09:45:30 ← **bez zmian** (DATETIME jest "głupie")
- ts → 2026-04-14 07:45:30 ← **zmniejszone o 2 godziny** (bo pokazuje UTC)
- created_at, updated_at → też pokazane w UTC

Wniosek:

- DATETIME przechowuje dokładnie to, co wpiszesz — nie konwertuje.
- TIMESTAMP zawsze przechowuje w **UTC** wewnętrznie i automatycznie konwertuje przy odczycie do aktualnej strefy sesji.

MySQL wyświetla wartości TIME w formacie: hh:mm:ss lub (dla dużych godzin): hhh:mm:ss

Zakres wartości: -838:59:59 do 838:59:59. **Z częścią ułamkową** (do 6 cyfr po przecinku): od '-838:59:59.000000' do '838:59:59.000000'

To jest jedna z najczęstszych pułapek w MySQL:

Wartość, którą wpisujesz	Jak MySQL to zinterpretuje	Komentarz
'11:12'	'11:12:00'	Z dwukropkiem = godzina:minuta
'1112' lub 1112	'00:11:12'	Bez dwukropka = minuty:sekundy
'12' lub 12	'00:00:12'	Tylko sekundy
'111'	'00:01:11'	minuty + sekundy

Zasada:

- Jeśli jest **dwukropek (:)** → traktuje jako **czas dnia** (godziny:minuty).
- Jeśli **nie ma dwukropka** → traktuje jako **czas upływający**, gdzie dwie ostatnie cyfry to sekundy.

Część ułamkowa (fractional seconds)

Od MySQL 5.6.4 możesz używać mikrosekund:

```
TIME          -- dokładność do sekundy
TIME(0)       -- to samo co powyżej
TIME(3)       -- milisekundy (3 cyfry po kropce)
TIME(6)       -- mikrosekundy (maksymalna precyzja)
```

Przykład:

```
CREATE TABLE test_time (
  id INT PRIMARY KEY,
```

```
t1 TIME,  
t2 TIME(6)  
);
```

```
INSERT INTO test_time VALUES (1, '15:30:45.123456', '15:30:45.123456');
```

YEAR to bardzo mały typ danych (zajmuje tylko **1 bajt**), służący wyłącznie do przechowywania roku.

Podstawowe informacje

- **Rozmiar:** 1 bajt
- **Zakres:** od **1901** do **2155** + wartość **0000**
- **Format wyświetlania:** zawsze **YYYY** (4 cyfry), np. 2025, 1999, 0000
- **Deklaracja:**
 - Zalecana: YEAR (bez liczby)
 - Przestarzała: YEAR(4) ← **nie używaj** (będzie usunięta w przyszłych wersjach MySQL)

Uwaga ważna: Od kilku lat MySQL odradza używanie YEAR(4). Zamiast tego używaj po prostu YEAR.

Jakie wartości można wstawiać?

MySQL akceptuje rok w kilku różnych formatach:

Sposób podania	Przykład	Jak MySQL to zrozumie	Komentarz
4-cyfrowy string	'2025', '1999'	2025, 1999	Najbezpieczniejszy
4-cyfrowa liczba	2025, 1999	2025, 1999	OK
1 lub 2 cyfry (string)	'5', '25', '69'	2005, 2025, 2069	00–69 → 2000–2069
1 lub 2 cyfry (string)	'70', '99'	1970, 1999	70–99 → 1970–1999
1 lub 2 cyfry (liczba)	5, 25, 69	2005, 2025, 2069	1–69 → 2001–2069
1 lub 2 cyfry (liczba)	70, 99	1970, 1999	70–99 → 1970–1999

Specjalne zachowanie przy zerze:

- 0 (jako **liczba**) → zostaje zapisane jako **0000**
- '0' lub '00' (jako **string**) → zostaje zapisane jako **2000**

Przykłady

```
CREATE TABLE test_year (  
  id      INT AUTO_INCREMENT PRIMARY KEY,  
  rok     YEAR,                -- poprawne  
  -- rok4  YEAR(4)             -- przestarzałe, nie używaj  
);
```

```
INSERT INTO test_year (rok) VALUES  
(2025),      -- OK → 2025  
('2025'),    -- OK → 2025  
(25),        -- liczba → 2025  
('25'),     -- string → 2025  
(69),        -- → 2069  
(70),        -- → 1970  
(0),         -- → 0000  
('0'),      -- → 2000 ← ważne!  
('00');     -- → 2000
```

Zachowanie przy niepoprawnych wartościach

- **Tryb nieściśły** (bez STRICT): Nieprawidłowy rok (np. 1800, 3000, 'abc') → zamieniany na **0000** + warning
- **Tryb ściśły** (STRICT_TRANS_TABLES lub STRICT_ALL_TABLES): Nieprawidłowy rok → **błąd** (insert się nie uda)

MySQL oferuje następujące typy danych do przechowywania tekstu i danych binarnych:

Typ	Pełna nazwa	Przeznaczenie	Maks. długość
CHAR	Character	Tekst o stałej długości	do 255 znaków
VARCHAR	Variable Character	Tekst o zmiennej długości	do 65 535 znaków
BINARY	Binary	Dane binarne o stałej długości	do 255 bajtów
VARBINARY	Variable Binary	Dane binarne o zmiennej długości	do 65 535 bajtów
BLOB	Binary Large Object	Duże obiekty binarne (obrazy, pliki itp.)	do 4 GB
TEXT	Text	Duży tekst (artykuły, opisy itp.)	do 4 GB
ENUM	Enumeration	Lista predefiniowanych wartości	do 65 535 wartości
SET	Set	Zbiór wielu wartości z listy	do 64 wartości

- **Typy znakowe** (CHAR, VARCHAR, TEXT, ENUM, SET) → Długość podaje się w **znakach** (character units). Uwzględniają zestaw znaków (np. utf8mb4 = do 4 bajtów na znak).

- **Typy binarne** (BINARY, VARBINARY, BLOB) → Długość podaje się w **bajtach** (byte units). Nie mają kodowania znaków.

Określanie zestawu znaków i sortowania (Character Set & Collation)

Możesz określić dla kolumny:

```
column_name VARCHAR(50)
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_polish_ci
```

- CHARACTER SET (lub skrót CHARSET) – definiuje kodowanie znaków
- COLLATE – definiuje reguły porównywania i sortowania
- CHARSET binary → zamienia typ znakowy na binarny (zobacz niżej)

Ważne zachowanie przy CHARACTER SET binary:

```
VARCHAR(10) CHARACTER SET binary    → staje się VARBINARY(10)
TEXT CHARACTER SET binary            → staje się BLOB
ENUM('a','b','c') CHARACTER SET binary → pozostaje ENUM (nie zmienia się na binarny!)
```

CHAR

- Stała długość: CHAR(M) → M od 0 do 255 znaków
- Dopełniany spacjami z prawej strony podczas zapisu
- Przy odczycie spacje są **usuwane** (chyba że włączysz tryb PAD_CHAR_TO_FULL_LENGTH)
- CHAR(0) jest dozwolone – zajmuje 1 bit, może przechowywać tylko NULL lub pusty string "

VARCHAR

- Zmienna długość: VARCHAR(M) → M od 0 do 65 535 znaków
- Przechowuje długość + dane (1 lub 2 bajty na prefix długości)
- **Nie usuwa** końcowych spacji (zgodne ze standardem SQL)
- Rzeczywista maksymalna długość zależy od kodowania i limitu wiersza (65 535 bajtów na cały wiersz)

BINARY i VARBINARY

- Analogiczne do CHAR i VARCHAR, ale binarne (w bajtach)
- BINARY(M) – max 255 bajtów
- VARBINARY(M) – max 65 535 bajtów

ENUM

- Może zawierać **tylko jedną** wartość z listy
- Maks. 65 535 różnych wartości
- Przechowywany wewnętrznie jako liczba całkowita

ENUM (Enumeration) to specjalny typ stringowy, który pozwala kolumnie przyjmować **tylko jedną wartość** z zapamiętej zdefiniowanej listy.

Zalety typu ENUM

- Bardzo **oszczędny** pod względem miejsca (przechowywany jako liczba, nie jako tekst).
- Zapytania i wyniki są **czytelne** – MySQL automatycznie zamienia liczby z powrotem na tekst.
- Wbudowana walidacja – nie da się wstawić wartości spoza listy (w trybie strict).

Przykład oszczędności miejsca: Jeśli wstawisz 1 milion wierszy z wartością 'medium':

- ENUM → zajmuje **1 milion bajtów**
- VARCHAR → zajęłoby **6 milionów bajtów**

Składnia

```
CREATE TABLE shirts (
  name VARCHAR(40),
  size ENUM('x-small', 'small', 'medium', 'large', 'x-large')
);
```

Każda wartość z listy ma przypisany **indeks** (numer):

Wartość	Indeks
NULL	NULL
" (pusty)	0
'x-small'	1
'small'	2
'medium'	3
'large'	4
'x-large'	5

- Maksymalna liczba elementów w ENUM: **65 535**
- Przy pobieraniu w kontekście numerycznym zwracany jest indeks (size + 0 zwraca liczbę).

Ważne pułapki i ograniczenia

1. **Nie używaj wartości wyglądających jak liczby!** To najczęstsze źródło błędów.

```
numbers ENUM('0', '1', '2')    -- źle!
```

- Wstawienie 2 → zostanie zapisane jako '1' (bo 2 to indeks)
- Wstawienie '2' → zostanie zapisane jako '2'
- Wstawienie '3' → zostanie zapisane jako '2' (indeks 3 = wartość '2')

2. **Kolejność sortowania** ENUM sortuje się **według indeksu**, a nie alfabetycznie! Przykład:
ENUM('b', 'a') → 'b' pojawi się przed 'a'.

Rozwiązania:

- Układaj listę alfabetycznie przy tworzeniu
- Lub sortuj jawnie: ORDER BY CAST(size AS CHAR) lub ORDER BY CONCAT(size)

3. Puste i NULL wartości

- Jeśli wstawiś niepoprawną wartość → wstawiana jest pusty string "" (indeks 0) jako błąd.
- W trybie **strict** → błąd zamiast wstawienia "".
- Jeśli kolumna zezwala na NULL → domyślną wartością jest NULL.
- Jeśli NOT NULL → domyślną wartością jest **pierwszy** element z listy.

4. Inne ograniczenia

- Nie możesz używać wyrażeń ani zmiennych przy definiowaniu listy:
ENUM('small', CONCAT('med', 'ium'), 'large') -- **NIE DZIAŁA**
- Duplikaty w liście → warning lub błąd (w strict mode).

SET

- Może zawierać **wiele** wartości z listy jednocześnie
- Maks. **64** różnych wartości
- Przechowywany jako bity (bardzo oszczędny)

SET to specjalny typ stringowy, który pozwala kolumnie przechowywać **zero, jedną lub wiele wartości** jednocześnie, wybranych z listy zdefiniowanej przy tworzeniu tabeli.

Wartości multiple są oddzielone przecinkiem (,). **Ważne:** Same elementy listy nie mogą zawierać przecinka.

Przykład

```
CREATE TABLE user_hobbies (  
    user_id INT,  
    hobbies SET('reading', 'sports', 'music', 'travel', 'gaming', 'cooking')  
);
```

```
INSERT INTO user_hobbies (user_id, hobbies) VALUES (2, 'reading,sports,music');
INSERT INTO user_hobbies (user_id, hobbies) VALUES (1, 'reading');
```

Kolumna hobbies może przyjmować następujące wartości:

- '' (pusty zestaw)
- 'reading'
- 'sports,music'
- 'reading,travel,gaming'
- 'music,reading,sports' (kolejność nie ma znaczenia przy wstawianiu)

Maksymalna liczba elementów: 64 różne wartości.

MySQL przechowuje wartości SET jako **liczbę** (bitową maskę):

- Każdy element listy ma przypisaną pozycję bitu (od najmniej znaczącego).
- Pierwszy element = bit 0 (wartość 1)
- Drugi element = bit 1 (wartość 2)
- Trzeci element = bit 2 (wartość 4)
- itd.

Przykład:

SET('a', 'b', 'c', 'd')

Element	Wartość dziesiętna	Wartość binarna
'a'	1	0001
'b'	2	0010
'c'	4	0100
'd'	8	1000

Jeśli wstawisz liczbę 9 (binarnie 1001), MySQL wybierze elementy 'a' i 'd' → wynik: 'a,d'.

Kluczowe zachowania SET

1. **Kolejność przy wstawianiu nie ma znaczenia** Przy odczycie wartości zawsze są zwracane w **kolejności z definicji** tabeli i każdy element pojawia się tylko raz.

```
INSERT INTO myset (col) VALUES ('a,d'), ('d,a'), ('a,d,d'), ('d,a,d');
-- Wszystkie zostaną wyświetlone jako 'a,d'
```

2. Niepoprawne wartości

- a. W trybie **nieściśłym**: niepoprawna wartość jest ignorowana + warning „Data truncated”
- b. W trybie **strict**: próba wstawienia niepoprawnej wartości kończy się **błędem**

3. **Sortowanie** Wartości SET są sortowane **numerycznie** (według wartości bitowej), a nie alfabetycznie. NULL sortuje się przed wszystkimi innymi wartościami.
4. **Wyświetlanie** Wartości są wyświetlane z wielkością liter taką, jaką podano przy tworzeniu tabeli.

Jak wyszukiwać dane w kolumnie SET?

Najlepsze i najbezpieczniejsze sposoby:

-- Najlepszy sposób

```
SELECT * FROM user_hobbies
WHERE FIND_IN_SET('reading', hobbies) > 0;
```

-- Alternatywa (działa, ale mniej precyzyjna)

```
SELECT * FROM user_hobbies
WHERE hobbies LIKE '%reading%';
```

-- Wyszukiwanie po masce bitowej (szybkie)

```
SELECT * FROM user_hobbies
WHERE hobbies & 1;                -- zawiera pierwszy element ('reading')
```

-- Dokładne dopasowanie całego zestawu

```
SELECT * FROM user_hobbies
WHERE hobbies = 'reading,travel';
```

Lekcja

Temat: Typy danych przestrzennych (Spatial Data Types), typ danych JSON w MySQL

Typy danych przestrzennych i funkcje są dostępne dla tabel korzystających z silników: MyISAM, InnoDB, NDB oraz ARCHIVE.

Typy danych przestrzennych (Spatial Data Types) w MySQL to specjalne typy kolumn, które pozwalają przechowywać, zarządzać i analizować **dane geometryczne / geograficzne** bezpośrednio w bazie danych.

Dzięki nim możesz w prosty sposób zapisywać np.:

- współrzędne GPS punktów (np. lokalizacja sklepu, użytkownika, sensora),
- trasy (linie),
- obszary (np. granice działki, strefy dostawy, poligony miast),
- kolekcje tych obiektów.

MySQL bazuje na standardzie **OpenGIS** (Simple Features), więc typy są zgodne z tym, co używają inne systemy GIS (np. PostGIS, Oracle Spatial, QGIS).

1. Podstawowe typy pojedyncze (Single Geometry)

Typ	Co przechowuje?	Przykład użycia
GEOMETRY	Dowolny rodzaj geometrii (uniwersalny typ)	Najczęściej używany, gdy nie wiesz z góry co będzie
POINT	Pojedynczy punkt (X, Y)	Lokalizacja użytkownika, adres, sensor
LINESTRING	Linia / łamana składająca się z wielu punktów	Trasa drogi, ścieżka rowerowa, granica rzeki
POLYGON	Zamknięty obszar (wielokąt) – może mieć „dziury”	Granice działki, strefa dostawy, jezioro

2. Typy kolekcyjne (Collections / Multi-*)

Typ	Co przechowuje?
MULTIPOINT	Zbiór wielu punktów
MULTILINESTRING	Zbiór wielu linii
MULTIPOLYGON	Zbiór wielu poligonów
GEOMETRYCOLLECTION	Mieszany zbiór dowolnych geometrii (punkty + linie + poligony)

3. Najważniejszy atrybut: SRID (Spatial Reference Identifier)

- To numer identyfikujący **system współrzędnych**.
- Najpopularniejszy: **SRID 4326** → to standard **WGS 84** (używany przez GPS, Google Maps, OpenStreetMap). Współrzędne to szerokość i długość geograficzna.
- SRID 0 → płaski układ kartezjański (zwykle X, Y bez jednostek geograficznych).

W **MySQL 8+** kolumna bez zdefiniowanego SRID nadal akceptuje dowolne dane geometryczne i możliwe jest utworzenie indeksu przestrzennego, jednak brak SRID oznacza brak kontroli nad układem współrzędnych oraz ryzyko niepoprawnych wyników operacji przestrzennych (np. obliczania odległości czy relacji typu „zawiera” lub „przecina”). Z kolei zastosowanie SRID (np. **GEOMETRY NOT NULL SRID 4326**) zapewnia spójność danych, ponieważ wszystkie geometrie muszą być zapisane w tym samym układzie odniesienia, co pozwala na poprawne i wiarygodne wykonywanie operacji przestrzennych oraz efektywne wykorzystanie indeksów przestrzennych.

<https://sqlize.online/sql/mysql80/7f085527e2df63413ebc5d0df135f641/>

Przykład

```

CREATE TABLE miejsca (
    id INT PRIMARY KEY,
    nazwa VARCHAR(100),
    lokalizacja POINT NOT NULL,      -- punkt GPS
    obszar POLYGON NOT NULL         -- obszar
);

-- Indeks przestrzenny (bardzo przyspiesza zapytania)
CREATE SPATIAL INDEX idx_lokalizacja ON miejsca(lokalizacja);

INSERT INTO miejsca (id, nazwa, lokalizacja)
VALUES (
    1,
    'Test',
    ST_GeomFromText('POINT(10 20)', 4326)
);

```

Zastosowanie

- Aplikacje lokalizacyjne („znajdź sklepy w promieniu 5 km”)
- Systemy logistyczne i nawigacyjne
- Analiza geograficzna („które działki przecina nowa droga?”)
- Aplikacje IoT (lokalizacja urządzeń)
- Mapy, GIS, smart city

Przykład

```

CREATE TABLE obiekty_przestrzenne (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nazwa VARCHAR(150) NOT NULL,
    typ ENUM('punkt', 'trasa', 'obszar', 'dowolny') NOT NULL,
    punkt POINT NOT NULL,          -- SRID 0 (kartezjański)
    trasa LINESTRING NOT NULL,     -- SRID 4326
    obszar POLYGON NOT NULL,       -- SRID 4326
    geometria GEOMETRY NOT NULL,   -- dowolny typ, SRID 4326
    opis TEXT NULL,
    data_dodania DATETIME DEFAULT CURRENT_TIMESTAMP,
    -- Indeksy przestrzenne (bardzo ważne dla szybkich zapytań!)
    SPATIAL INDEX idx_punkt (punkt),
    SPATIAL INDEX idx_trasa (trasa),
    SPATIAL INDEX idx_obszar (obszar),
    SPATIAL INDEX idx_geometria (geometria)
) ENGINE=InnoDB;

```

Przykłady wstawiania danych (INSERT)

-- 1. Typ: punkt

```
INSERT INTO obiekty_przestrzenne (nazwa, typ, punkt, trasa, obszar, geometria)
VALUES (
    'Testowy punkt - Pałac Kultury',
    'punkt',
    ST_GeomFromText('POINT(21.0122 52.2297)', 0),          -- SRID 0 (kartezjański)
    ST_GeomFromText('LINESTRING(21.0122 52.2297, 21.0150 52.2320)', 4326),
    ST_GeomFromText('POLYGON((21.010 52.228, 21.014 52.228, 21.014 52.231, 21.010 52.231,
21.010 52.228))', 4326),
    ST_GeomFromText('POINT(21.0122 52.2297)', 4326)
);
```

-- 2. Typ: trasa

```
INSERT INTO obiekty_przestrzenne (nazwa, typ, punkt, trasa, obszar, geometria)
VALUES (
    'Trasa Warszawa-Łódź',
    'trasa',
    ST_GeomFromText('POINT(21.0122 52.2297)', 0),
    ST_GeomFromText('LINESTRING(21.0122 52.2297, 21.1000 52.3000, 21.4000 52.6000)', 4326),
    ST_GeomFromText('POLYGON((21.000 52.200, 21.050 52.200, 21.050 52.230, 21.000 52.230,
21.000 52.200))', 4326),
    ST_GeomFromText('LINESTRING(21.0122 52.2297, 21.4000 52.6000)', 4326)
);
```

-- 3. Typ: obszar

```
INSERT INTO obiekty_przestrzenne (nazwa, typ, punkt, trasa, obszar, geometria)
VALUES (
    'Park Łazienkowski - obszar',
    'obszar',
    ST_GeomFromText('POINT(21.0345 52.2148)', 0),
    ST_GeomFromText('LINESTRING(21.030 52.210, 21.040 52.220)', 4326),
    ST_GeomFromText('POLYGON((21.030 52.210, 21.040 52.210, 21.040 52.220, 21.030 52.220,
21.030 52.210))', 4326),
    ST_GeomFromText('POLYGON((21.030 52.210, 21.040 52.210, 21.040 52.220, 21.030 52.220,
21.030 52.210))', 4326)
);
```

-- 4. Typ: dowolny

```
INSERT INTO obiekty_przestrzenne (nazwa, typ, punkt, trasa, obszar, geometria, opis)
VALUES (
    'Dowolny obiekt testowy',
    'dowolny',
    ST_GeomFromText('POINT(10 20)', 0),
    ST_GeomFromText('LINESTRING(10 20, 30 40)', 4326),
    ST_GeomFromText('POLYGON((10 20, 30 20, 30 40, 10 40, 10 20))', 4326),
    ST_GeomFromText('POINT(21.015 52.232)', 4326),
    'Przykład obiektu dowolnego typu - wszystkie geometrie wypełnione'
);
```

ST_GeomFromText() to najważniejsza i najczęściej używana funkcja w MySQL przy pracy z danymi przestrzennymi. **Jest to funkcja, która zamienia tekst (w formacie WKT) na**

prawdziwy obiekt geometryczny (POINT, LINESTRING, POLYGON itp.), który MySQL potrafi zapisać w kolumnie typu przestrzennego.

WKT (Well-Known Text)

To **standardowy sposób zapisywania geometrii jako zwykły tekst**. Jest czytelny dla człowieka.

Najpopularniejsze przykłady WKT:

Geometria	Przykład WKT	Co oznacza
POINT	POINT(52.2297 21.0122)	Pojedynczy punkt (szerokość, długość)
LINESTRING	LINESTRING(52.23 21.01, 52.24 21.05, 52.25 21.10)	Linia przechodząca przez 3 punkty
POLYGON	POLYGON((52.20 21.00, 52.22 21.00, 52.22 21.05, 52.20 21.05, 52.20 21.00))	Zamknięty obszar (czworokąt)
POLYGON z dziurą	POLYGON((zewnątrzny), (wewnętrzny))	Obszar z "dziurą" w środku

Uwaga: Kolejność współrzędnych to zawsze (szerokość geograficzna, długość geograficzna) przy SRID 4326.

SPATIAL INDEX (indeks przestrzenny) to specjalny rodzaj indeksu, który pozwala MySQL **bardzo szybko** wyszukiwać dane geometryczne według ich położenia na mapie.

Dzięki niemu zapytania typu:

- „Znajdź wszystkie sklepy w promieniu 5 km ode mnie”
- „Które działki przecina nowa droga?”
- „Znajdź wszystkie punkty wewnątrz tego obszaru”

działają błyskawicznie, zamiast skanować całą tabelę wiersz po wierszu.

Najważniejsze zasady SPATIAL INDEX w MySQL

1. **Działa tylko na kolumnach przestrzennych** (POINT, LINESTRING, POLYGON, GEOMETRY itd.)
2. **Kolumna musi mieć SRID** i być NOT NULL

- a. Bez tego indeks SPATIAL **nie zadziała** lub będzie ignorowany.
- 3. **Najlepiej działa na InnoDB (8.0+)**
 - a. MyISAM też obsługuje, ale InnoDB jest zdecydowanie zalecany.
- 4. Indeks SPATIAL używa algorytmu **R-Tree** (drzewo R), które jest zoptymalizowane pod dane przestrzenne.

Przykład:

Tu zadziała mySQL wersja 8:

<https://sqlize.online/sql/mysql80/c04d1554f7f2eb49a94311cef197b7a9/>

```
CREATE TABLE obiekty_przestrzenne (  
  id          INT PRIMARY KEY AUTO_INCREMENT,  
  nazwa       VARCHAR(150) NOT NULL,  
  
  punkt       POINT NOT NULL SRID 4326,      -- musi być NOT NULL + SRID  
  trasa       LINESTRING NOT NULL SRID 4326,  
  obszar      POLYGON NOT NULL SRID 4326,  
  opis        TEXT NULL  
) ENGINE=InnoDB;  
  
-- Tworzenie indeksów przestrzennych  
CREATE SPATIAL INDEX idx_punkt  ON obiekty_przestrzenne(punkt);  
CREATE SPATIAL INDEX idx_trasa  ON obiekty_przestrzenne(trasa);  
CREATE SPATIAL INDEX idx_obszar ON obiekty_przestrzenne(obszar);
```

Najlepsza praktyka – twórz indeks już przy tworzeniu tabeli:

```
CREATE TABLE obiekty_przestrzenne (  
  ...  
  punkt  POINT NOT NULL SRID 4326,  
  trasa  LINESTRING NULL SRID 4326,  
  
  SPATIAL INDEX idx_punkt (punkt),  
  SPATIAL INDEX idx_trasa (trasa)  
) ENGINE=InnoDB;
```

Przykład zapytania, które korzysta z SPATIAL INDEX

```
-- Znajdź wszystkie punkty w promieniu 3 km od mojej lokalizacji  
SELECT nazwa,  
       ST_Distance_Sphere(punkt, ST_GeomFromText('POINT(52.2297 21.0122)',  
4326)) AS odleglosc_m  
FROM obiekty_przestrzenne  
WHERE ST_Distance_Sphere(punkt, ST_GeomFromText('POINT(52.2297 21.0122)',
```

```
4326)) <= 3000  
ORDER BY odleglosc_m;
```

Przykład 1: ST_Distance (odległość między punktami)

```
CREATE TABLE miejsca (  
    id INT PRIMARY KEY,  
    nazwa VARCHAR(100),  
    lokalizacja POINT NOT NULL );
```

```
INSERT INTO miejsca VALUES  
(1, 'A', POINT(52.2297, 21.0122)),  
(2, 'B', POINT(52.4064, 16.9252));
```

ST_Distance(punkt1, punkt2) - liczy zwykłą odległość w układzie X/Y (płaska płaszczyzna)

Czyli:

- traktuje punkty jak na kartce papieru 📄
- ignoruje kształt Ziemi 🌐
- nie uwzględnia szerokości geograficznej ani krzywizny

```
SELECT  
    a.nazwa AS punkt1, b.nazwa AS punkt2,  
    ST_Distance(a.lokalizacja, b.lokalizacja) AS odleglosc  
FROM miejsca a  
JOIN miejsca b ON a.id < b.id;
```

ST_Distance_Sphere - to liczy odległość w metrach na kuli ziemskiej.

```
SELECT  
    ST_Distance_Sphere( POINT(21.0122, 52.2297),  
        POINT(16.9252, 52.4064) ) AS distance_meters;
```

Przykład 1: Prawidłowa wersja (MySQL 8+ / geograficzna)

```
SELECT  
    ST_Distance_Sphere(  
        POINT(21.0122, 52.2297),  
        POINT(16.9252, 52.4064)  
    ) AS distance_meters;
```

Przykład 2

Znajdź punkty w promieni 5 km:

```
SELECT *  
FROM miejsca
```

```
WHERE ST_Distance_Sphere(  
    lokalizacja,  
    POINT(21.0122, 52.2297)  
) <= 5000;
```

W wersji (MariaDB 10.4)

Nie masz ST_Distance_Sphere, więc robisz:

```
SELECT *,  
ST_Distance(  
    lokalizacja,  
    POINT(21.0122, 52.2297)  
) AS dist  
FROM miejsca  
HAVING dist < 0.05;
```

To NIE są km ani metry, to tylko przybliżenie matematyczne

JSON to natywny typ danych dodany w **MySQL 5.7** (i mocno ulepszony w MySQL 8.0+). Pozwala przechowywać dane w formacie **JSON** (JavaScript Object Notation) bezpośrednio w kolumnie bazy danych.

Zamiast rozbijać dane na wiele kolumn lub tabel, możesz zapisać cały „obiekt” lub „tablicę” w jednej kolumnie.

Podstawowa składnia

```
CREATE TABLE produkty (  
    id          INT PRIMARY KEY AUTO_INCREMENT,  
    nazwa       VARCHAR(100) NOT NULL,  
    dane_json   JSON NOT NULL,           -- kolumna typu JSON  
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Zalety typu JSON w MySQL

Zaleta	Opis
Elastyczność	Możesz przechowywać różną strukturę danych w różnych wierszach
Czytelność	Dane są zapisane w formacie, który łatwo odczytać

Wydajność	MySQL od wersji 8.0 bardzo dobrze optymalizuje kolumny JSON
Funkcje wyszukiwania	Możesz wyszukiwać i filtrować po kluczach wewnątrz JSON
Walidacja	MySQL automatycznie sprawdza poprawność składni JSON

Przykłady użycia

1. Prosty INSERT

```
INSERT INTO produkty (nazwa, dane_json) VALUES
('iPhone 15',
  '{"marka": "Apple",
    "cena": 4299,
    "kolory": ["czarny", "niebieski", "różowy"],
    "specyfikacja": {
      "ekran": "6.1 cala",
      "pamiec": "128 GB",
      "aparat": "48 MP"
    },
    "promocja": true
  }'
);
```

2. Bardziej złożony przykład (zamówienie)

```
INSERT INTO zamowienia (nr_zamowienia, dane) VALUES
('ZAM-2026-0042',
  '{"klient": {
    "imie": "Anna",
    "nazwisko": "Kowalska",
    "adres": {"miasto": "Warszawa", "ulica": "Marszałkowska 10"}
  },
  "produkty": [
    {"id": 15, "nazwa": "Laptop", "ilosc": 1, "cena": 3499},
    {"id": 87, "nazwa": "Myszka", "ilosc": 2, "cena": 89}
  ],
  "status": "wysłane",
  "data_wyslki": "2026-04-10"
}'
);
```

Najważniejsze funkcje do pracy z JSON

Funkcja	Co robi
JSON_EXTRACT() lub ->	Wyciąga wartość po kluczu
JSON_UNQUOTE() lub ->>	Wyciąga wartość i usuwa cudzysłowy
JSON_CONTAINS()	Sprawdza czy zawiera daną wartość

JSON_ARRAY()	Tworzy tablicę JSON
JSON_OBJECT()	Tworzy obiekt JSON
JSON_SET()	Aktualizuje lub dodaje klucz
JSON_REMOVE()	Usuwa klucz
JSON_KEYS()	Zwraca listę kluczy

Przykłady zapytań:

SELECT

```

nazwa,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.marka')) AS marka,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.cena')) AS cena,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.specyfikacja.ekran')) AS ekran,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.promocja')) AS promocja
FROM produkty
LIMIT 0, 25;
```

Wersja z filtrowaniem (WHERE)

SELECT

```

nazwa,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.marka')) AS marka,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.cena')) AS cena
FROM produkty
WHERE JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.marka')) = 'Apple'
LIMIT 0, 25;
```

Wersja z filtrowaniem (WHERE)

SELECT

```

nazwa,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.marka')) AS marka,
JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.cena')) AS cena
FROM produkty
WHERE JSON_UNQUOTE(JSON_EXTRACT(dane_json, '$.marka')) = 'Apple'
LIMIT 0, 25;
```

Dodatkowe przydatne warianty wyszukiwania w tablicy:

-- Szukamy produktów, które mają kolor "czarny" LUB "niebieski"

```

SELECT nazwa
FROM produkty
WHERE JSON_CONTAINS(dane_json, '"czarny"', '$.kolory')
OR JSON_CONTAINS(dane_json, '"niebieski"', '$.kolory');
```

```
-- Szukamy produktów, które mają jednocześnie "czarny" i "niebieski"
SELECT nazwa
FROM produkty
WHERE JSON_CONTAINS(dane_json, '["czarny", "niebieski"]', '$.kolory');
```